SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS: **Total Marks: 30**

• A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

• An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

• A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Code of Conduct:

I Aaron Samuel S(192411022) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	**Weigh tage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution

		standards			
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. SOAP-Based Hospital Management Web Service

A SOAP (Simple Object Access Protocol) web service is suitable for a hospital management system because hospitals need structured, reliable, secure, and interoperable communication between departments such as reception, laboratory, pharmacy, billing, and medical records.

Basic architecture

Hospital Client/Department

|

| SOAP Request (XML)

v

SOAP Web Service

|

v

Hospital Database

|

| SOAP Response (XML)

v

Hospital Client/Department

Structure of a SOAP message

A SOAP message is an XML document with three major parts:

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

  <soap:Header>
    <!-- Security, authentication, transaction information -->
  </soap:Header>

  <soap:Body>
    <!-- Actual request or response data -->
  </soap:Body>

</soap:Envelope>
```

Envelope – The root element that identifies the document as a SOAP message. It contains the Header and Body.

Header – Optional section containing additional information such as authentication credentials, security tokens, transaction IDs, or routing information.

Body – Contains the actual application request or response, such as adding a patient or retrieving medical records.

Major operations

A hospital service could provide operations such as:

Operation	Purpose	Example input
addPatient	Register a new patient	Patient ID, name, DOB, contact
getPatient	Retrieve patient details	Patient ID
scheduleAppointment	Schedule an appointment	Patient ID, doctor ID, date/time
cancelAppointment	Cancel an appointment	Appointment ID
getMedicalRecords	Retrieve medical records	Patient ID
updateMedicalRecord	Update a patient's record	Patient ID, diagnosis, treatment
getAppointments	Retrieve appointments	Patient ID/date

Example: Add Patient request

```
<soap:Envelope
   xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
   xmlns:h="http://hospital.example.com/">

   <soap:Header>
      <h:Authentication>
         <h:UserID>doctor123</h:UserID>
         <h:Token>secure-token</h:Token>
      </h:Authentication>
   </soap:Header>

   <soap:Body>
      <h:addPatient>
         <h:patient>
            <h:patientId>P1001</h:patientId>
            <h:name>John Smith</h:name>
            <h:dateOfBirth>1990-05-15</h:dateOfBirth>
            <h:phone>9876543210</h:phone>
         </h:patient>
      </h:addPatient>
   </soap:Body>

</soap:Envelope>
```

The server validates the request, stores the patient information in the database, and returns a SOAP response:

```
<soap:Envelope
   xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

   <soap:Body>
      <addPatientResponse
         xmlns="http://hospital.example.com/">
         <patientId>P1001</patientId>
         <status>SUCCESS</status>
```

```
        <message>Patient registered successfully</message>
      </addPatientResponse>
   </soap:Body>

</soap:Envelope>
```

SOAP fault/error response

If the patient ID already exists, the server should not simply return an invalid response. It can return a SOAP Fault:

```
<soap:Fault>
   <faultcode>soap:Client</faultcode>
   <faultstring>Patient ID already exists</faultstring>
   <detail>
      <errorCode>PATIENT_001</errorCode>
   </detail>
</soap:Fault>
```

Why SOAP is appropriate

Reliability: SOAP supports standardized fault handling and can be combined with WS-* specifications for reliable messaging and transactions.

Security: SOAP can use WS-Security for authentication, integrity, confidentiality, and security tokens. HTTPS can additionally protect communication in transit.

Interoperability: XML-based SOAP can communicate between systems implemented using different programming languages and platforms.

Strict structure: WSDL and XML Schema allow message formats and data types to be formally defined.

Enterprise support: SOAP is well suited to complex enterprise environments requiring transactions, security policies, and standardized error handling.

For a hospital, where patient information is sensitive and communication failures can have serious consequences, these characteristics make SOAP a strong choice.

2. WSDL Design for an Enterprise Web Service

WSDL (Web Services Description Language) provides a machine-readable description of a web service. It tells external applications what operations are available, what data they accept and return, how messages are formatted, how communication occurs, and where the service is located.

A simplified WSDL structure is:

Definitions

├── Types

├── Messages

├── PortType

├── Binding

└── Service

    └── Port

1. Types

The types section defines the data structures used by the service, normally using XML Schema (XSD).

```
<types>
  <xsd:schema
    targetNamespace="http://example.com/customer">

    <xsd:element name="Customer">
      <xsd:complexType>
        <xsd:sequence>
          <xsd:element name="id" type="xsd:int"/>
          <xsd:element name="name" type="xsd:string"/>
          <xsd:element name="email" type="xsd:string"/>
        </xsd:sequence>
      </xsd:complexType>
    </xsd:element>

  </xsd:schema>
</types>
```

This ensures that different systems agree on the structure and data types.

2. Messages

A message defines the data exchanged between client and server.

For example:

```
<message name="GetCustomerRequest">
   <part name="customerId" type="xsd:int"/>
</message>

<message name="GetCustomerResponse">
   <part name="customer" element="tns:Customer"/>
</message>
```

Here:

GetCustomerRequest represents the request.
GetCustomerResponse represents the returned customer data.

3. PortType

The portType defines the abstract interface of the service—the operations that clients can invoke.

```
<portType name="CustomerPortType">

   <operation name="getCustomer">
      <input message="tns:GetCustomerRequest"/>
      <output message="tns:GetCustomerResponse"/>
   </operation>

   <operation name="updateCustomer">
      <input message="tns:UpdateCustomerRequest"/>
      <output message="tns:UpdateCustomerResponse"/>
   </operation>

</portType>
```

Thus, it tells the client what the service can do, without specifying the network protocol details.

4. Binding

The binding specifies how the abstract operations are actually transmitted. For example, the service may use SOAP over HTTP.

```
<binding name="CustomerSoapBinding"
        type="tns:CustomerPortType">

    <soap:binding
       style="document"
       transport="http://schemas.xmlsoap.org/soap/http"/>

    <operation name="getCustomer">
       <soap:operation
          soapAction="getCustomer"/>
    </operation>

    <operation name="updateCustomer">
       <soap:operation
          soapAction="updateCustomer"/>
    </operation>

</binding>
```

The binding therefore connects the abstract interface to a concrete communication protocol.

5. Service and Port

The service specifies where the web service can be accessed.

```
<service name="CustomerService">

    <port name="CustomerSoapPort"
         binding="tns:CustomerSoapBinding">

       <soap:address
          location="https://api.example.com/customer"/>
    </port>
```

```
</service>
```

The port combines a binding with an actual endpoint URL.

Overall WSDL skeleton

```
<definitions
   name="CustomerService"
   targetNamespace="http://example.com/customer"
   xmlns:tns="http://example.com/customer"
   xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
   xmlns:xsd="http://www.w3.org/2001/XMLSchema"
   xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/">

   <types>
      <!-- XML Schema data definitions -->
   </types>

   <message name="GetCustomerRequest">
      <!-- Request parameters -->
   </message>

   <message name="GetCustomerResponse">
      <!-- Response parameters -->
   </message>

   <portType name="CustomerPortType">
      <!-- Operations -->
   </portType>

   <binding name="CustomerSoapBinding"
            type="tns:CustomerPortType">
      <!-- SOAP communication details -->
   </binding>
```

```
  <service name="CustomerService">
    <port name="CustomerSoapPort"
          binding="tns:CustomerSoapBinding">
        <soap:address
          location="https://api.example.com/customer"/>
    </port>
  </service>

</definitions>
```

Importance of WSDL

WSDL acts as a contract between the service provider and service consumer.

For example:

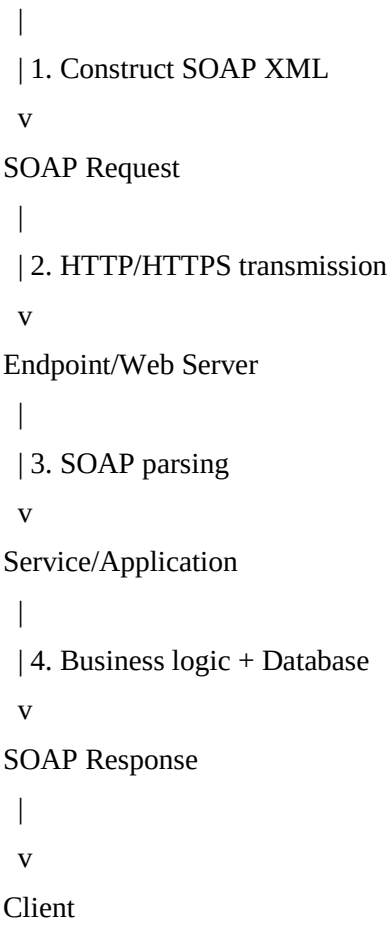
```
WSDL
 |
 +--> What data?       --> Types
 |
 +--> What messages?   --> Message
 |
 +--> What operations? --> PortType
 |
 +--> How communicate? --> Binding
 |
 +--> Where to access? --> Service/Port
```

This improves interoperability because an external system can obtain the WSDL and generate client-side code or construct valid SOAP requests without needing to know the internal implementation of the service.

3. Debugging a SOAP Communication System

When SOAP requests are not processed correctly or responses are delayed/incorrect, debugging should examine the complete communication path:

Client



An error at any stage can cause the observed problem.

Common problems

1. SOAP message formatting errors

SOAP requires well-formed XML. Problems include:

Missing closing tags.

Incorrect XML nesting.

Invalid characters.

Incorrect element names.

Missing required fields.

Incorrect data types.

Incorrect SOAP version.

For example, this is invalid:

<patient>
  <name>John</patient>
</name>

The XML parser may reject the entire request.

Solution: Validate the XML against the expected XML Schema and inspect the raw request and response.

2. Incorrect namespaces

Namespaces are particularly important in SOAP.

For example, the server may expect:

xmlns:h="http://hospital.example.com/patient"

but the client sends:

xmlns:h="http://example.com/patient"

Even though the element names look identical, they belong to different namespaces and may not be recognized by the service.

Solution:

Compare client namespaces with the WSDL.

Check the namespace of the SOAP Envelope.

Check operation and data-element namespaces.

Verify namespace prefixes and their associated URIs.

The prefix itself does not have to be identical, but the namespace URI must match.

3. Incorrect endpoint

The client might send the request to the wrong URL:

https://server.example.com/service

while the actual endpoint is:

https://api.example.com/hospitalService

This can result in HTTP 404/405 errors, connection failures, or requests reaching the wrong application.

Solution: Verify the endpoint in the WSDL, configuration files, API gateway, reverse proxy, and server deployment.

4. SOAP version mismatch

SOAP 1.1 and SOAP 1.2 use different namespaces and HTTP conventions.

For example, SOAP 1.1 commonly uses:

http://schemas.xmlsoap.org/soap/envelope/

while SOAP 1.2 uses:

http://www.w3.org/2003/05/soap-envelope

A client and server expecting different versions can fail to process messages correctly.

Solution: Confirm that the client, WSDL, server framework, and endpoint all use the same SOAP version.

5. Incorrect SOAPAction or HTTP headers

A SOAP 1.1 request may contain an HTTP header such as:

Content-Type: text/xml

SOAPAction: "getPatient"

If the SOAPAction does not correspond to the expected operation, the server may route the request incorrectly.

Solution: Compare HTTP headers against the WSDL and server configuration.

6. Authentication and security problems

If the service uses authentication or WS-Security, problems can occur because of:

Expired security tokens.

Invalid credentials.

Missing security headers.

Incorrect certificates.

Timestamp/clock differences.

Invalid signatures.

Solution: Inspect the SOAP Header and server security logs without exposing sensitive credentials.

7. Timeout and endpoint performance problems

Delayed responses can be caused by:

```
Client
  |
  +--> Network delay
  |
  +--> Proxy/API gateway delay
  |
  +--> Web server overload
  |
  +--> Slow application processing
  |
  +--> Database query delay
  |
  +--> External service delay
```

Solution: Measure the time spent at each stage and check server, application, database, and network logs.

Recommended debugging approach

A systematic procedure is preferable to changing multiple components simultaneously.

Step 1: Capture the raw request and response

Use a SOAP testing/debugging tool or HTTP trace to inspect:

HTTP method.
URL.
HTTP headers.
SOAP Envelope.
SOAP Header.
SOAP Body.
Response status code.
Response SOAP Fault.
Never log passwords, security tokens, or medical information unnecessarily.

Step 2: Validate against WSDL/XSD
Check that:

SOAP request
↓
Correct Envelope?
↓
Correct namespace?
↓
Correct operation?
↓
Correct parameters?
↓
Correct data types?
↓
Valid according to XSD?

Step 3: Check the endpoint
Verify:

DNS resolution.
Network connectivity.
Port availability.
HTTPS/TLS configuration.

Endpoint URL.

Firewall/proxy/API gateway configuration.

Step 4: Check server logs

Look for:

XML parsing errors.

Namespace errors.

Authentication failures.

Application exceptions.

Database errors.

Timeout messages.

Incorrect operation routing.

Step 5: Test with a known-good request

Send a simple request copied from the WSDL or a working test case. If that succeeds, compare it with the failing request and identify the difference.

Step 6: Check SOAP Fault handling

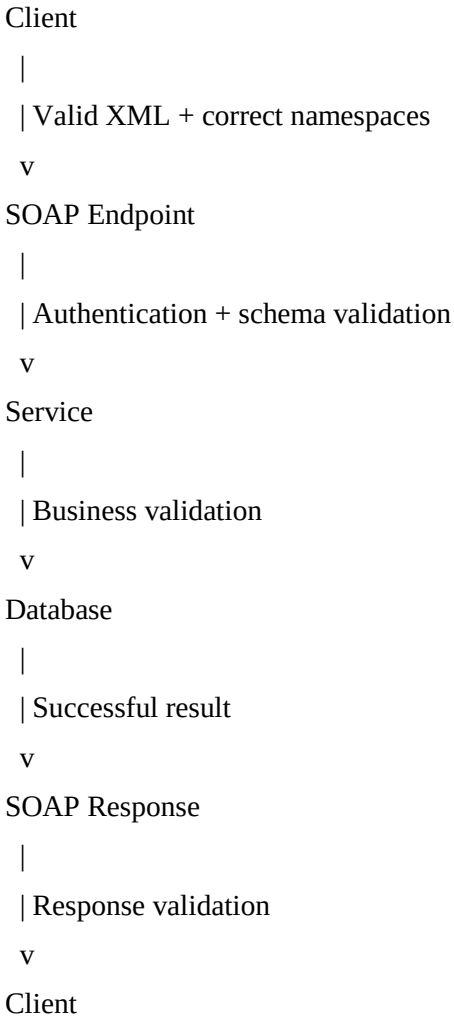
The client should explicitly process SOAP Faults rather than treating every response as successful.

```
if HTTP/SOAP response contains Fault:
    read fault code
    read fault reason
    inspect detail
    log correlation ID
    handle/retry where appropriate
else:
    validate response
    process business result
```

Transient failures such as temporary network or service unavailability may be retried with controlled backoff. Permanent errors such as invalid input should normally not be repeatedly retried.

How the corrected system provides reliable communication

After debugging, the communication process should look like:



Reliability is achieved through WSDL-based contract validation, correct namespaces and SOAP versions, endpoint verification, secure authentication, structured SOAP Faults, logging/correlation IDs, timeouts, and carefully controlled retries.

Together, these measures ensure that both the client and server have a consistent understanding of the message format, operation, communication protocol, and error conditions, making the SOAP system much more dependable.